EC2 Networking

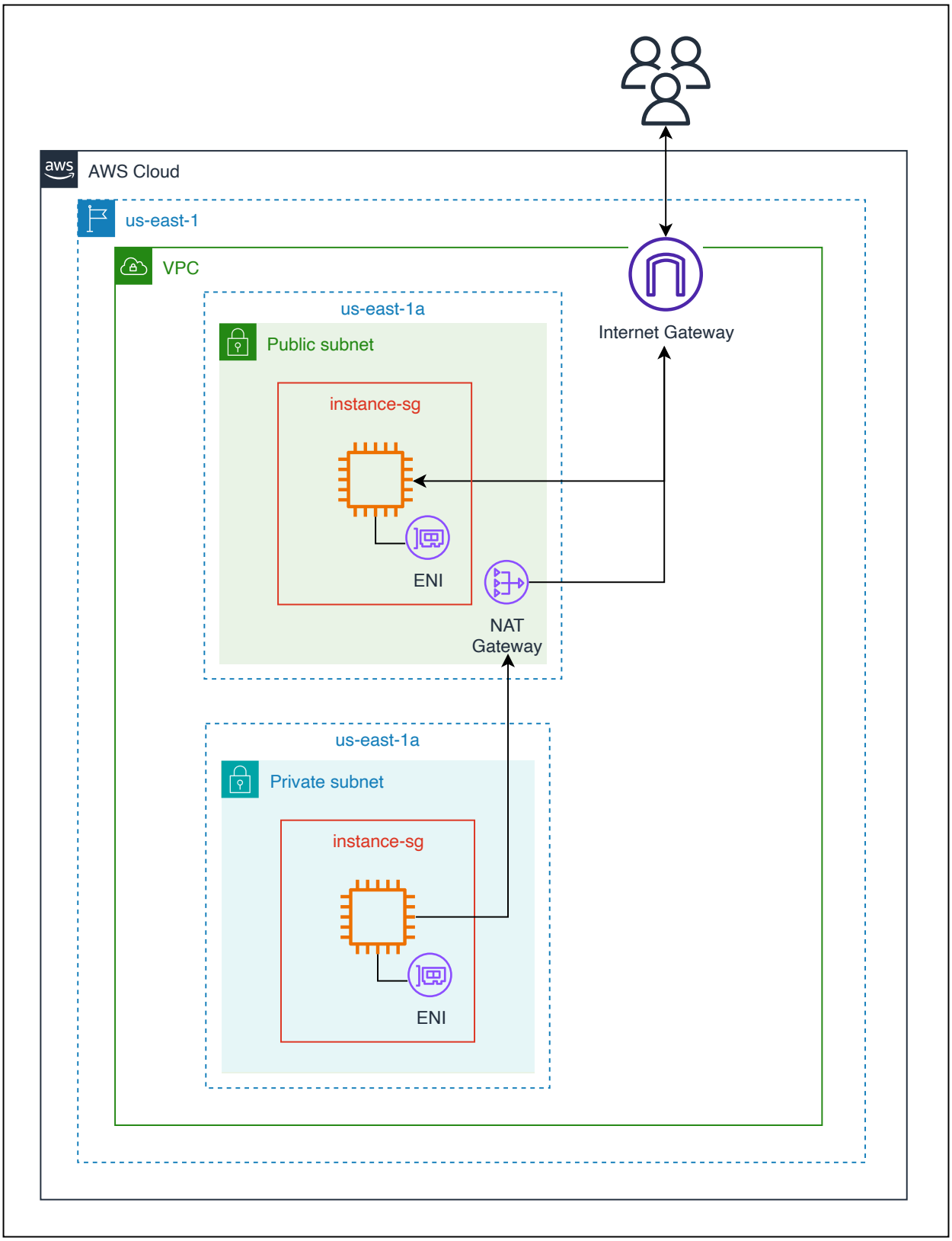
Explore how to configure Amazon EC2 networking covering VPCs, public and private subnets, elastic network interfaces, and IP addressing options. Understand how to secure instances using stateful security groups and best practices for inbound and outbound traffic control. This lesson equips you with the knowledge to design flexible and secure network architectures for cloud applications.

Amazon EC2 instances operate within a Virtual Private Cloud (VPC), a logically isolated network in AWS. Within a VPC, we define subnets, which segment our IP address space. These subnets can be either public or private, depending on whether they are connected to an internet gateway.

EC2 instances in public subnets can communicate with the internet if assigned a public IP and properly configured with security groups and route tables. Those in private subnets are typically isolated from the internet and access external services via NAT gateways or instances.

To facilitate internal and external communication, every EC2 instance is associated with an elastic network interface (ENI), which provides network connectivity and manages IP addressing.

Ask



EC2 instances in private and public subnets

Ask

IP addressing: Public, private, and elastic IPs

When launching EC2 instances, we deal with three types of IPs:

- Private IPs are always assigned. We use private IPs for communication within our VPC (e.g., between tiers of an app).
- Public IPs are assigned optionally. These are ephemeral and change if the instance is stopped. These are used for short-term or non-critical internet connectivity.
- Elastic IPs (EIPs) are static public IPs allocated to our AWS account and manually associated with instances or ENIs. EIPs are ideal for applications requiring fixed IP addresses, such as DNS records or IP whitelisting. However, AWS charges for unused EIPs, so make sure to release any that are no longer needed.

Static vs. dynamic IP address

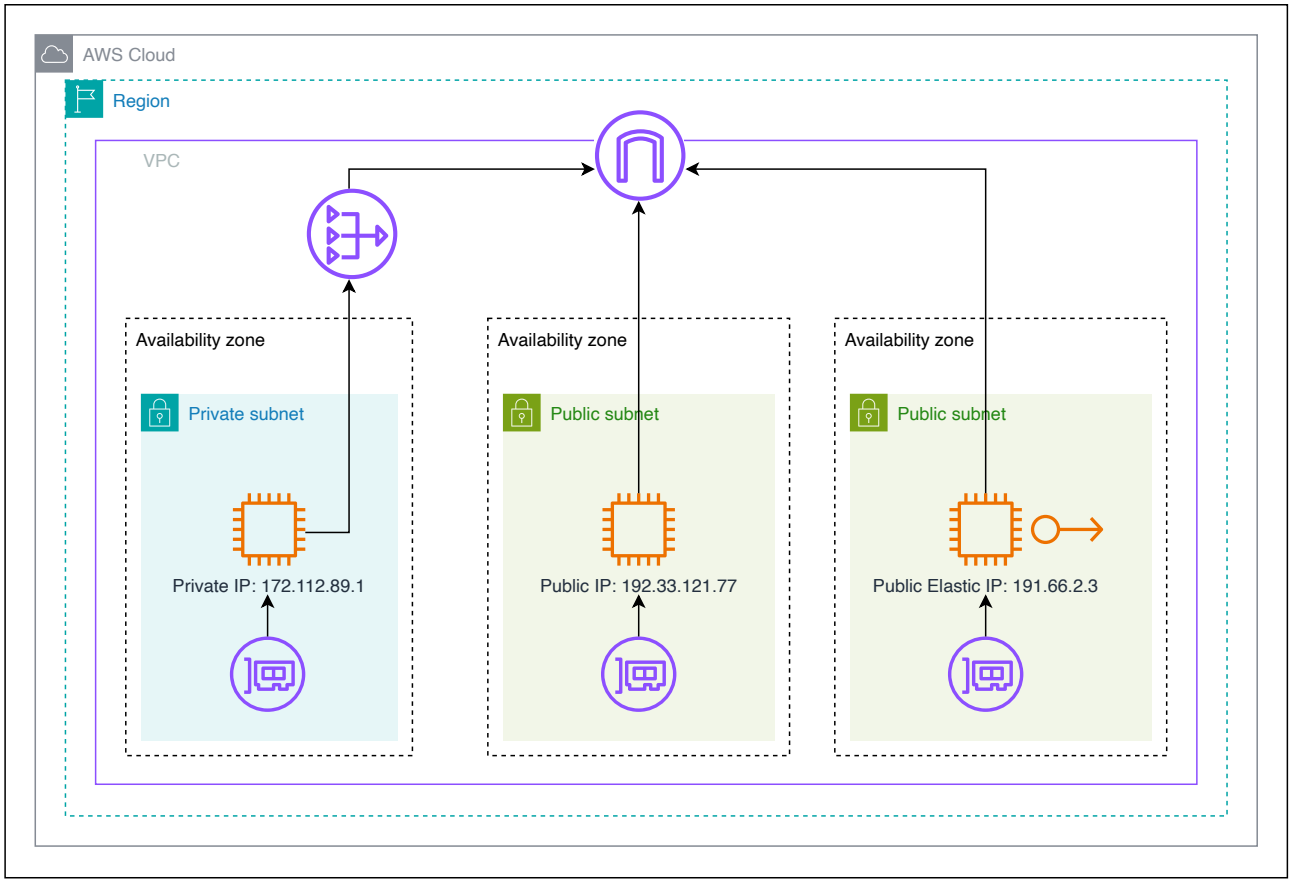
When launching EC2 instances, one key decision is whether to use a dynamic or static public IP address. This decision affects not only how an application communicates externally, but also how it integrates with DNS, firewalls, and client dependencies. Let's look at some of the features of static and dynamic IPs:

Category	Dynamic IPs	Static IPs (Elastic IPs)
Default Behavior	Assigned by default to instances in public subnets (if enabled)	Must be explicitly allocated and associated
Persistence	Ephemeral—changes on instance stop/start	Persistent—remains the same across restarts or instance replacements
Ideal Use Cases	• Testing environments • Short-lived instances (e.g., CI/CD runners) • Auto Scaling behind ELB	• IP-whitelisted APIs • Fast propagation needed for custom DNS • VPNs, bastion hosts
Integration Flexibility	Less predictable—requires DNS updates or dynamic resolution	Reliable IP—prevents breakage in external systems relying on a fixed IP

Ask

DNS Use	Best for internal or frequently updated DNS environments	Best when the DNS propagation delay must be minimized

We can also assign multiple IPs to a single instance using elastic network interfaces (ENIs). Additionally, we can also move a network interface between instances to support failover and high availability. The diagram below illustrates the distinction between ENIs and Elastic IP addresses (EIPs):



1 / 3

Initial state

Ask

In this example, we have three EC2 instances: one in a private subnet and two in public subnets.

- The instance in the private subnet is attached to an ENI only.
- One of the public instances is also associated with an ENI only.
- The second public instance, however, is associated with both an ENI and an EIP.

When these instances are rebooted, only the instance with the EIP retains its public IP address. The others, which rely on automatically assigned public IPs, receive new ones upon reboot. This highlights the key benefit of using an EIP when persistent public addressing is required.

> DNS resolution is automatically provided inside the VPC via AmazonProvidedDNS, enabling seamless access to internal AWS resources and external domains.

Securing instances with security groups

Security groups are stateful firewalls that control traffic to and from EC2 instances. Rules can be configured for specific protocols, port ranges, and sources (IP ranges or other SGs). Take a look at an example of a security group that allows HTTPS and SSH traffic:

- We permit TCP traffic on port `443` from `0.0.0.0/0`.
- For SSH, we allow access for administrators only, restricting the TCP port `22` to a known IP range like `203.0.113.10/32`.

Ask

Security group

Security groups apply least-privilege access, meaning we should only open what is explicitly needed. This minimizes our attack surface and enforces security best practices.

Referencing security groups in rules

Rather than specifying IP ranges, AWS allows us to reference another security group as a traffic source or destination. This is particularly powerful in dynamic environments where instances scale in and out.

Let's assume we have three security groups for three tiers in our setup:

- **SG-A:** Web servers
- **SG-B:** Application servers
- **SG-C:** Databases

Interactions between security groups and resources

We can create a rule in SG-B allowing inbound TCP traffic on port `8080` from SG-A. This means any instance associated with SG-A can send traffic to instances associated with SG-B without worrying about changing IP addresses.

Here's what a sample security group rule that allows access to another security group looks like:

Ask

index.json Node.js

```
{
  "FromPort": 3306,
  "ToPort": 3306,
  "IpProtocol": "tcp",
  "UserIdGroupPairs": [{"GroupId": "<Valid_Security_Group_ID>"}]
}
```

Security group in JSON format

Security groups are stateful, so if an inbound rule allows traffic, the response is automatically permitted. This simplifies configuration. By default, inbound traffic is denied and outbound traffic is allowed. However, organizations may choose to explicitly restrict outbound traffic for better control.

Common misconfigurations to avoid

When working with security groups, keep these in mind:

- **Overlapping or redundant rules:** These can result from having multiple rules allowing broad access without a purpose.
- **Overly permissive outbound rules:** By default, all outbound traffic is allowed. Some organizations restrict this to enforce stricter egress controls.
- **Allowing `0.0.0.0/0` on sensitive ports:** Opening all IPs for ports like SSH (22) without IP restrictions invites unnecessary exposure.

Careful design and frequent audits of security group rules help prevent security vulnerabilities.

Ask

Conclusion

We've now seen how networking in Amazon EC2 is built around flexible, scalable, and secure configurations using the foundational building blocks of AWS: VPCs, subnets, IP addressing, and security groups. From dynamically launching a single EC2 instance to managing a production-grade, multi-tier application, understanding EC2 networking allows us to:

- Confidently configure network access using public and private subnets, ENIs, and static vs. dynamic IPs.

Ask